

PHI / PII De-Identification Gateway

Project Documentation

1. Project Overview

The **PHI / PII De-Identification Gateway** is designed to remove sensitive information from clinical notes and medical documents before they are sent to Large Language Models (LLMs).

The main purpose is simple: clinical information can still be used for tasks such as summarization or analysis, while personally identifiable information is protected.

The gateway identifies sensitive data such as patient names, phone numbers, email addresses, SSNs, MRNs, IP addresses, dates, locations, and other identifiers covered under the **18 HIPAA Safe Harbor categories**.

2. Main Features

2.1 PHI / PII Detection and Redaction

The system detects and replaces sensitive information from clinical text.

Some of the identifiers handled by the gateway include:

- Names
- Social Security Numbers (SSNs)
- Phone numbers
- Email addresses
- Medical Record Numbers (MRNs)
- IP addresses
- Dates
- Locations
- Other HIPAA Safe Harbor identifiers

For example:

Original:

```
Patient John Doe (SSN: 999-88-7777)
```

After de-identification:

```
Patient [NAME_1] (SSN: [SSN_1])
```

This allows the clinical text to be processed without directly exposing the patient's personal information.

2.2 Date Shifting

The system uses **HMAC-seeded date shifting** to change dates in the clinical text.

The important point is that the relationship between dates is maintained.

For example, if one medical event happened before another event in the original note, the shifted dates will keep the same order.

The date can be shifted within a range of **±364 days**.

This helps protect the actual dates while keeping the timeline useful for analysis.

2.3 Age Capping

The gateway also handles ages above 89.

Instead of keeping the exact age, the system converts it into:

[AGE: 90+]

This follows the HIPAA age-capping requirement and avoids exposing the exact age of older patients.

2.4 Context-Aware Eponym Protection

Some medical terms contain names of people but are not actually patient or provider names.

For example:

- Parkinson's disease
- Foley catheter

The system uses **context-aware analysis** to distinguish these medical terms from actual names.

This helps avoid unnecessary redaction while still protecting real patient and provider names.

2.5 Secure Rehydration

After sensitive information is replaced, the original values are stored in an encrypted mapping.

The system uses **AES-128 Fernet encryption** to protect this mapping.

This allows an LLM to work with the de-identified text. If the response needs to be mapped back to the original patient information, the encrypted mapping can be used to restore the original values.

In simple terms:

Original clinical text → De-identification → LLM processing → Rehydration → Original values restored

The original sensitive information is therefore not directly passed to the LLM.

2.6 Web UI

The project also includes a simple interactive web interface.

The UI provides:

- Side-by-side text editing
- Sample text loading
- PDF upload
- Instant text parsing

This makes it easier to test the de-identification process without directly working with the API.

3. Project Structure

The project is organised into separate modules so that each part has a specific responsibility.

```
phi-deidentification-gateway/  
  
├─ APPROACH_NOTE.md  
├─ FAILURES.md  
├─ README.md  
├─ requirements.txt  
  
├─ src/  
│   ├── api.py  
│   ├── config.py  
│   ├── date_shifter.py  
│   ├── eponym_whitelist.py  
│   ├── eval_harness.py  
│   ├── gateway.py  
│   ├── hybrid_analyzer.py  
│   ├── ner_model.py  
│   ├── pseudonymizer.py  
│   └── rehydrator.py  
  
├─ tests/  
│   ├── test_extended_cases.py  
│   └── test_gateway.py
```

```
|   └─ test_hybrid.py

|   └─ scripts/
|       ├── baseline_presidio.py
|       ├── baseline_regex.py
|       ├── demo_roundtrip.py
|       ├── generate_synthetic_notes.py
|       └─ train_bio_clinical_ner.py

└─ data/
    ├── samples/
    ├── eval/
    └─ key.bin
```

The main source files handle the API, HIPAA categories, date processing, eponym detection, benchmarking, PHI detection, pseudonymization, and encrypted rehydration.

4. Important Source Files

`api.py`

Contains the **FastAPI REST API** endpoints used to communicate with the gateway.

`config.py`

Contains the definitions for the **18 HIPAA Safe Harbor categories**.

`date_shifter.py`

Handles date shifting and age capping.

`eponym_whitelist.py`

Handles medical eponyms and helps distinguish them from actual names.

`eval_harness.py`

Used to measure system performance such as accuracy, recall, and latency.

`gateway.py`

Provides the main gateway interface for **de-identification and rehydration**.

`hybrid_analyzer.py`

Combines different detection methods, mainly **Regex and Transformer NER**.

`ner_model.py`

Handles entity detection using **spaCy** and **BioClinical-ModernBERT**.

`pseudonymizer.py`

Handles the replacement of sensitive information with pseudonyms.

`rehydrator.py`

Manages the encrypted mapping and restoration of the original information.

5. Testing Files

The project also contains separate test files for checking different parts of the system.

`test_extended_cases.py`

Used for testing edge cases and checking for information leakage.

`test_gateway.py`

Tests the main gateway functionality.

`test_hybrid.py`

Tests the hybrid detection system and eponym handling.

6. Running the Project

Step 1 – Install Dependencies

Make sure Python is installed and run:

```
pip install -r requirements.txt
```

This installs the packages required by the project.

Step 2 – Run the Tests

To run the automated test suite:

```
python -m pytest
```

This is used to check the system and verify the de-identification behaviour.

Step 3 – Run the Demo

The complete de-identification and rehydration flow can be tested using:

```
python scripts/demo_roundtrip.py
```

This provides an end-to-end demonstration of how the system processes the text and restores the original values.

Step 4 – Start the API and Web UI

The FastAPI server can be started using:

```
python -m uvicorn src.api:app --host 127.0.0.1 --port 8080
```

After starting the server, the Web UI can be accessed locally at:

```
http://127.0.0.1:8080/
```

The API documentation is available through the FastAPI OpenAPI page at:

```
http://127.0.0.1:8080/docs
```

7. API Specification

The gateway provides two main API endpoints:

- `POST /deidentify`
 - `POST /rehydrate`
-

7.1 `/deidentify`

This endpoint receives clinical text and a patient seed.

Input

```
{
  "text": "Patient John Doe (SSN: 999-88-7777) admitted on 2024-02-11.",
  "patient_seed": 42
}
```

The system identifies the sensitive information and replaces it with safe placeholders.

Output

```
{
  "masked_text": "Patient [NAME_1] (SSN: [SSN_1]) admitted on [DATE_1].",
  "encrypted_mapping": "gAAAAABqiHmRV_djfk893rCH3HZeXBx8U98AeXyJILohe..."
}
```

The `masked_text` contains the de-identified version.

The `encrypted_mapping` contains the protected mapping required for restoring the original values later.

8. `/rehydrate`

After the de-identified text has been processed by an LLM, the response can be sent to the `/rehydrate` endpoint.

Input

```
{
  "llm_response": "Summary: Patient [NAME_1] (SSN: [SSN_1]) admitted on [DATE_1].",
  "encrypted_mapping": "gAAAAABqiHmRV_djfk893rCH3HZeXBx8U98AeXyJILohe..."
}
```

The endpoint uses the encrypted mapping to replace the placeholders with their original values.

Output

```
{
  "restored_text": "Summary: Patient John Doe (SSN: 999-88-7777) admitted on 2024-02-11."
}
```

The final output contains the restored information from the original clinical text.

9. Overall Workflow

The complete workflow of the gateway can be understood in four steps:

Step 1 – Input

Clinical text containing PHI / PII is provided to the gateway.

↓

Step 2 – De-identification

The gateway detects sensitive information and replaces it with placeholders such as `[NAME_1]`, `[SSN_1]`, and `[DATE_1]`.

↓

Step 3 – LLM Processing

The cleaned text can be sent to an LLM for tasks such as summarization or analysis.

↓

Step 4 – Rehydration

If required, the LLM response can be passed back to the gateway. The encrypted mapping is used to restore the original values.

This approach keeps the sensitive information separated from the text sent for LLM processing.

10. Conclusion

The PHI / PII De-Identification Gateway provides a way to process clinical text while reducing direct exposure of sensitive patient information.

The project combines several components, including **Regex**, **spaCy NER**, **BioClinical-ModernBERT**, **date shifting**, **age capping**, **eponym protection**, **pseudonymization**, and **encrypted rehydration**.

The system can be used through both a **FastAPI REST API** and a simple Web UI.

The two main API operations are `/deidentify` for removing sensitive information and `/rehydrate` for restoring the original values when required.

The project structure also keeps detection, encryption, testing, benchmarking, and API functionality in separate modules, making the system easier to test and maintain.